Project: Preparing for Cloud Native

Due: 01/31/2026 11:59 PM
Extended Deadline: 02/02/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Your Very Own LLM Inference Server
- Task Overview
- Task

Infrastructure as Code
- Task Overview
- Task

k8s Primitives
- Task Overview
- Task

Wrap Up
- Conclusion

Your Very Own LLM Inference Server (Task)

Introduction

In this preparation for Cloud Native project, you will be given a Flask project which you have to make changes to in order to build your LLM inference server. Please accept the GitHub Classroom invite and clone the project to your development environment.

System Architecture

At the end of this project, you will have a system architecture similar to the diagram shown below:

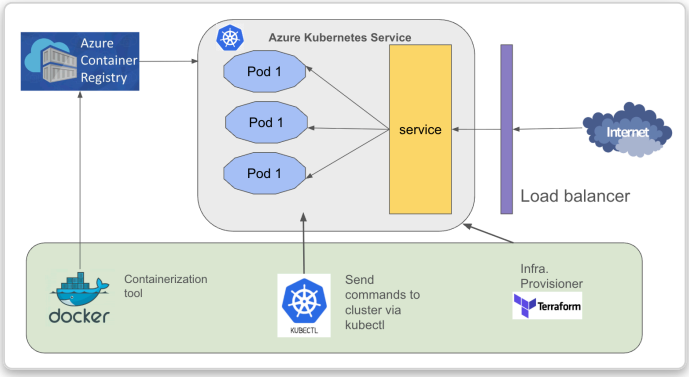

Figure: Final Architecture Diagram

Take some time to review the diagram and familiarize yourself with each component. Understanding how these pieces work together will make it much easier to follow the remaining steps of the project.

What's in the Source Code

For this task, your focus will be on the `flask-app` directory.

Change directory into the flask-app directory. This directory will hold all code related to the LLM inference server you will be developing.
The following is a tree structure of the flask-app directory:

```
├── docker
│   └── Dockerfile
├── llm_server
│   ├── __init__.py
│   └── main.py
├── Makefile
└── pyproject.toml
```

Please spend some few minutes going through each file/directory so you get a sense of what we'll be working on.

The pyproject.toml file

The `pyproject.toml` file is a standardized configuration file that defines how a Python project is **built**, **installed**, and **managed**. It was introduced to replace older files such as `requirements.txt, setup.py, and setup.cfg`, providing a modern, unified, and tool-independent way to build and package Python applications.

Sub-task1: Update the pyproject.toml file

- `name`: set the name to llm_server
- `version`: 1.0.0
- `description`: come up with your own description of the application you're building
- `dependencies`: Your application needs the following dependencies: flask, llama-cpp-python, and waitress

A Short Note on Application Dependencies

- **Flask** – A simple and easy-to-use Python framework that lets you create web applications and APIs.
- **llama-cpp-python** – A tool that lets your app run large language models (LLaMA) locally on your computer.

Waitress – A reliable program that runs your Flask app on a web server so other people can access it.

By updating all the `TODOs` in the pyproject.toml file, you know you're through with sub-task1.

Sub-task2: Setting up your Dev Environment

You'll use a **virtual environment** to develop this application.
A virtual environment creates an isolated workspace for your project, keeping its dependencies separate from the global Python installation. This prevents version conflicts between different projects and makes it easier to reproduce your development setup on other machines.

First, install the virtual environment tool by running the following command:

```
sudo apt install python3.12-venv
```

Next, you'll install the build tools on your VM.

```
sudo apt-get install -y build-essential
```

`build-essential` and `cmake` provide the core tools needed to compile and build software from source on Ubuntu. They're essential in cloud-native development for installing dependencies, building native modules, and ensuring consistent builds across environments

Finally, you'll set up your virtual environment and start coding your way through the project. Run these commands in the `flask-app` directory:

```
python3 -m venv .venv
source .venv/bin/activate
pip install flit
flit install --symlink
```

All the above commands do is:

1. Create a `virtual environment` where dependencies will be installed in isolation
2. Activate the virtual environment.
3. Install the `flit build package` which is a Python packaging tool that lets you build and install Python packages without needing a setup.py file
4. Finally, Flit builds your package and copies it into your Python environment's site-packages folder. The --symlink allows you to make real-time changes to your source code without having to manually run the flit install command anytime your code changes.

Running your Code

To run your code, use the following command:

`flask-app`

```
 * Serving Flask app 'llm_server.main'
 * Debug mode: on
WARNING: This is a development server.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8000
 * Running on http://172.26.14.206:8000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 122-044-935
```

Output after a successful run of your application

As you can see from the above logs, there's a warning about not to use this in production.

We'd need to use a production-grade server to serve our LLM application. Luckily, we have already installed the `waitress` package from sub-task1. Unlike Flask's built-in development server, Waitress is designed to handle real web traffic **reliably** and **concurrently**.

Press CTRL + C on your keyboard to stop the Flask server.
To run the application using waitress, use the following command:

```
waitress-serve --listen=0.0.0.0:8000 ll

#Output
INFO:waitress:Serving on http://0.0.0.0
```

As you can see, the warning message has “disappeared”. That means “waitress” is serving our application.

In your terminal make a request to your endpoint by running the following:

```
curl http://127.0.0.1:8000
```

You should get the output: “Yay! The app starts.....“ If you’re working on a virtual machine, you can expose the port 8000, go to your browser, enter http:<YOUR_VM_PUBLIC_IP>:8000.

You should see the **“Yay! The app starts.....”**

Sub-task3: Implement the LLM Server API Route

In sub-task2, you got your application running. However, we haven’t discussed the contents of the `llm_server/main.py` file. The **main.py** file is where you implement your API endpoints.

Remember from the previous sub-task, when you go to http://<ip-address>:8000, you get the response: “Yay! The app starts…”. The implementation for that route is located in the **main.py** file with the following code:

```
@app.route("/")
def index():
    return "Yay! The app starts....."
```

Following your manager's wish for a personal LLM inference server, you begin development by checking out [the tutorial you bookmarked earlier](), which contains code written by the Data Science intern from last Fall. The intern has no knowledge of Flask or other Python Web frameworks and only

implemented the LLM inference in a Python Notebook file.

Your task is straightforward, create an API endpoint for your LLM inference server using the intern's code as starting point.

After reading through the intern's code, you want to build a minimum viable inference server for TinyLlama-1B model using Flask, the standard WSGI web-server that is easy to work with. You want your system to do the following:

Take in a user's message.
Process the message to get the output for what the large language model is saying.
Return that message to the user.

Precisely, you want your server to have the following routes:

`/healthcheck`, which will return the string "OK". When you have CPU-intensive tasks like ML-model inference, you want to have simple probes that allow everyone using the container to know when it's able to get more traffic or not. A simple route like this is useful for that purpose.
`/api?message=<message>`, which will take a user message, add it to the tutorial's prompt `template`, and return the large language model's response deterministically (setting `temperature` to `0` and `max_tokens` to `None`). Specifically, you need to return the entire object `llama-cpp-python` gives you when you provide the model input and other parameters, **not just the output text**.
Finally, remove the `index` route. Real pros deploy in silence — just like the dark web.

You should implement the above in the `main.py` file.

Information

[HINTS]

You will need to download the `model(tinyllama-1.1b-chat-`

`v1.0.Q2_K.gguf)` file into the `flask-app` directory.

This sub-task shouldn't take you too long to complete. If you are taking longer than necessary, seek help via our discussion forum.

Interim Validation

Once you're done implementing the endpoints, you should run your application using the following command:

```
waitress-serve --listen=0.0.0.0:8000 ll
```

Before proceeding, we would like to give you some simple tests to run to make sure that you have a working implementation of the server. After running your server following the above command, the following commands should return responses similar to the ones demonstrated. In particular, pay close attention to the "choices" field and the "model" field:

```
$curl http://127.0.0.1:8000/healthcheck
127.0.0.1 - - [29/Jan/2024 20:52:48] "G
OK
----------------------------------------
$curl -G --data-urlencode 'message=what

{
  "choices": [
    {
      "finish_reason": "stop",
      "index": 0,
      "logprobs": null,
      "text": "\nYes, 1 + 1 is the answ
    }
  ],
  "created": 1761062441,
  "id": "cmpl-eb48e167-2b0f-47b7-87f8-e
  "model": "tinyllama-1.1b-chat-v1.0.Q2
  "object": "text_completion",
  "usage": { "completion_tokens": 16, "
}
```

If you're accessing your server via the browser, you can use the following URLs:

```
http://<YOUR-VM-IP>:8000/healthcheck
http://<YOUR-VM-IP>:8000/api?message=hello
```

Productionizing your LLM Inference Server

Your application works just fine. However, you want to share it with the developer next to you to try it out on their local computer. The developer doesn't care about the source code or whatever, all they need is some kind of file that they can run directly.

As you may be aware, currently, the way to run the application is to

get the source code,
install flit,
run flit install –symlink, and finally
run the waitress command: waitress-serve --listen=0.0.0.0:8000 llm_server.main:app.

While this approach works, it is not an efficient way of doing things. Instead, you package your app as a Python wheel and share the wheel file as a distribution package so others can quickly install it on their machines and run it.

To generate a wheel file for our inference server, we run the following command:

```
pip wheel ./ -w dist/
```

If this command runs successfully, you should see the "**Successfully built llm_server**" in your console output. Make sure you're in the `flask-app` directory before running the above command. The contents of the `dist` directory should be similar to what we have below:

```
├── blinker-1.9.0-py3-none-any.whl
├── click-8.3.0-py3-none-any.whl
├── diskcache-5.6.3-py3-none-any.whl
├── flask-3.1.2-py3-none-any.whl
├── itsdangerous-2.2.0-py3-none-any.whl
├── jinja2-3.1.6-py3-none-any.whl
```

```
├── llama_cpp_python-0.3.16-cp313-cp31
├── llm_server-1.0.0-py2.py3-none-any.v
├── llm_server-1.0.0.tar.gz
├── markupsafe-3.0.3-cp313-cp313-macos
├── numpy-2.3.4-cp313-cp313-macosx_14_(
├── typing_extensions-4.15.0-py3-none-a
├── waitress-3.0.2-py3-none-any.whl
└── werkzeug-3.1.3-py3-none-any.whl
```

The file of interest to us is the `llm_server-1.0.0-py2.py3-none-any.whl.`

Next, you will install and run this file using the following commands:

```
pip install dist/llm_server-1.0.0-py2.p
waitress-serve --listen=0.0.0.0:8000 ll
```

Your wheel file only consists of the source code and application dependencies. It does not contain the model file, **tinyllama-1.1b-chat-v1.0.Q2_K.gguf**. That means anytime you're sharing the wheel file, you must attach the model file as well.

> **Warning**
>
> Keep in mind that whenever you **modify** your source code, you need to rebuild the wheel file to apply and see those changes.

You've successfully completed this sub-task when accessing the **/healthcheck** and **/api?message=hello** endpoints returns the expected responses.

Sub-task 4: Containerize your Application

After reviewing your initial work, your boss is very pleased with your progress. She initially asks you to deploy the application on a virtual machine and call it a day. However, she then recalls a recent conference she attended on scalability, reliability, and resilience. One key takeaway from that event was that containers are the modern, efficient way to package and deploy applications. She now wants you to

explore **containerizing the LLM inference server** using **Docker**.

Install Docker

Docker is the tool you'll use to package your application into a container image.
The Ubuntu VM you're using doesn't come with Docker preinstalled, so you'll need to set it up yourself.

This time, we're not handing you the exact commands. it's your turn to explore!
Search online for how to install and configure Docker on Ubuntu. By the end, you should have:

- Docker installed on your VM
- Docker configured to run without needing sudo every time

The Dockerfile
There's an empty Dockerfile in the `flask-app/docker` directory. You will write your own docker statements that do the following:

- Use base-image `python:3.12-alpine`
- Installs the packages required by **llama-cpp-python** by running `apk update && apk add gcc libc-dev g++`
- Copies both your `wheel file` and the `model file` to the container.
- Install your wheel file.
- Run the appropriate waitress command to start your application

The above task shouldn't take too long to complete. As always, if you're taking too long to get it done, make a post on our discussion forum. Let us know where you are stuck and how we can help.

Build your Docker Container
To build your docker container, run the following command:

```
docker image build -f docker/Dockerfile
```

Run you container image

```
docker run -it -p 8000:8000 llmapp:late
```

Final Validation

Visit http://<vm-ip>:8000 in your browser. If you receive a response from the API server, your container is running successfully. You can further verify this by testing the /healthcheck and /api?message=hi endpoints to ensure your LLM inference server is returning the expected responses.

Submit your Work

Before submitting your work, make sure that:

- you have built the docker container image.
- your image is tagged exactly, `llmapp:latest`, as the submitter will look out for this image during grading.
- your latest code has been pushed to GitHub. Remember to use this same repository for all Cloud-Native projects.

Submitter Tools

The submitter depends on the following Ubuntu packages. Please install them with this command:

```
sudo apt install -y jq tree curl
```

Download the Submitter

In your project's **root directory**, use the following commands to download and submit your work:

```
wget https://cloudnativecourse.blob.cor
```

```
chmod +x submitter
./submitter -t task1
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform